

自我改进系统的可证伪发布门：规模化下的恒定不变量

Deepak Soni AI

架构师deepak.satna@gmail.com

2026 年 7 月

摘要

自我改进智能体运行时的安全声明几乎总是自我评定的：一份策略文件、一个护栏机制、README 中的一句承诺。本文描述可证伪发布门，一种方法论，其中每一项新能力在发布前必须通过预先声明、机器可检查的验收套件，同时一组固定的恒定不变量在每一道门中得以保留。我们在开放运行时 Antah.karan.a 中实例化该方法，然后做了方法论文只有通过实践才能证明的事：我们跟随同一运行时成长，并追问这些保证是否存续。安全关键性质——即任何动作未经控制环铸造的能力令牌不得到达效应器——在有界模型的可达状态上被机器穷尽检查；一个故意破坏的模型产生最短反例，因此检查器显然具有效力。随后我们将该运行时推进了六个后续版本。在每一个版本中，动作安全不变量 inv-1 至 inv-6 均保持不变，无需任何修改，其中一个版本新增三项能力而未引入任何新不变量。在同样的效力纪律下，又增加了六个机器检查的族：具有可证明遗忘的内存、受治理的智能体、后量子记录上的校准弃权、多子智能体的测试框架、自我改进循环本身，以及其产物的驻留性。验收套件从 122 项测试增长至 563 项。承重结果位于负空间：在能力翻倍以上的过程中，安全核心既未被削弱也未被重新设计。最后的族是首次在真实硬件上运行：门控自我改进将一个小模型从 20% 准确率提升至 70%，同时自动拒绝一个仅虚增置信度的候选，整个受治理路径每次请求成本为 0.021 毫秒，占模型推理的 0.008%。我们发布运行时、工具和门套件；每个数字均可通过单一命令复现。

1 引言

一个能够改变自身行为的智能体之所以有用，是因为它不是静态的；而危险也正源于此。困难的问题不在于“这一个版本是否安全？”，而在于“当智能体运行时获得记忆、工具访问、多智能体协调以及修改自身策略的能力时，每一个新能力是否都保持了上一个能力的安全性 [20, 25, 26]”。如今，这个问题即便有答案，也往往是通过人工检查和良好意图来回答的 [1]。系统的作者声称存在护栏机制，而其证据是系统对安全性进行了自我评估。我们认为，自我改进系统需要的是一个安全流程，而非一次性的审计，并且这个流程应当是可证伪的。我们的建议是可证伪的发布门禁。系统获得的每一项能力在发布时都必须通过一道门禁：一个预先声明、可由机器检查的验收套件，

该套件必须在代码被视为存在之前通过。一组少量的常设不变量在所有门禁之间得到维护，因此后续能力无法破坏先前能力的保证。安全关键核心是经过机器检查的（在模型上验证），而非仅仅被断言。

我们通过构建一个运行时系统 Antah.karan.a，沿着六级门禁的阶梯（图 1）具体地发展了这一方法：从根据迹重建任何过去决策的能力（G7），到对抗性工具使用集成（G8）、无漂移学习（G9）、集群规模治理（G10）、多租户隔离（G11），最后到一个自治循环，其不可绕过属性经过机器检查，且其自我修改在构造上受到约束（G12）。每一级台阶的选择都旨在使下一级台阶的构建变得安全。在能够证明在一个时间步内控制整个集群之前，不应尝试构建自我改进循环。在门禁能够在不放松自身的情况下进行学习之前，不应尝试构建自我改进循环。

The advancement ladder, end to end: the proven action-safety core,
then preserved and carried outward across six releases

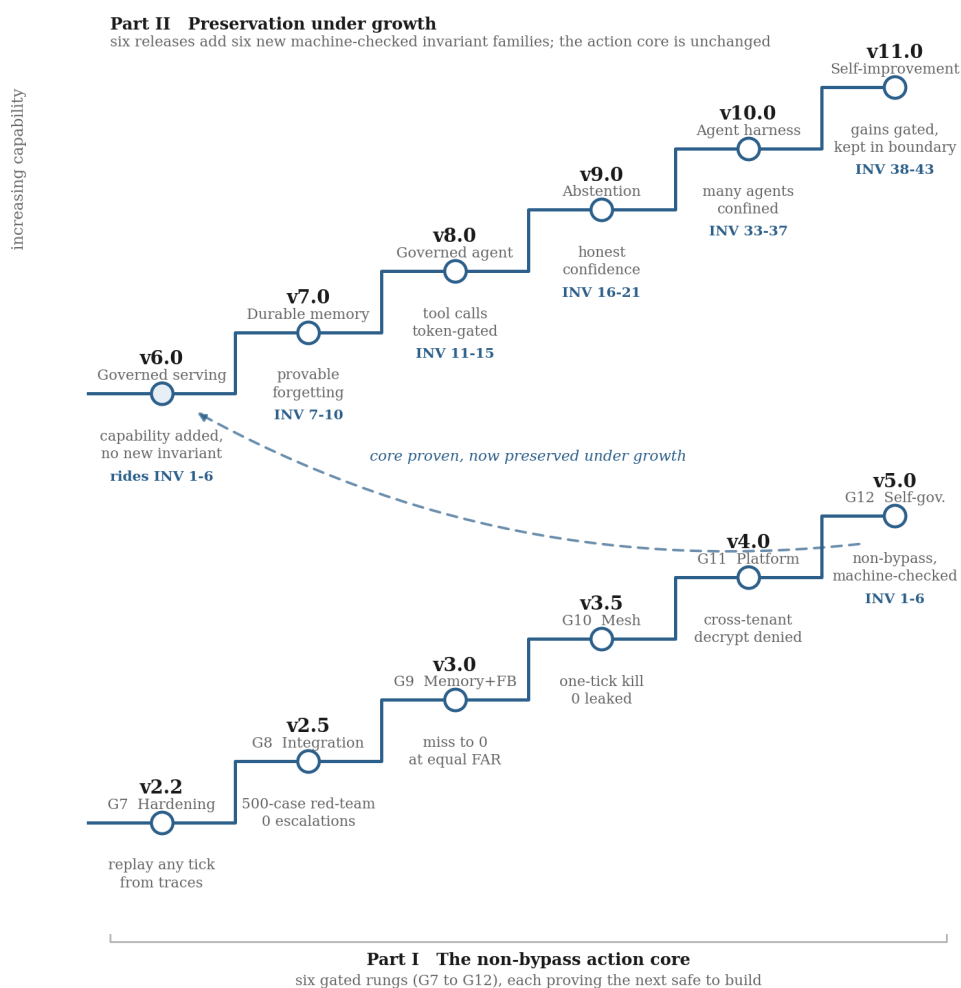


图 1：端到端的进阶阶梯。第一部分（下层）是经过验证的动作安全核心：从 G7 到 G12 的六级门禁台阶，每一级都证明下一级构建的安全性，其中不可绕过属性在 G12 处经过机器检查（inv-1 至 inv-6）。第二部分（上层）将同一核心贯穿于六个进一步的发布版本；每一级台阶都标明了其引入的不变量族，受治理的服务（v6.0）被标记为在增加能力的同时未引入新的常设不变量。核心在整个攀登过程中保持不变。

然而，在演示瞬间冻结的系统上展示的方法，仅被证明能成功一次。有趣的论断并非不变量在梯子最初搭建的那一刻成立过一次，而是它们能否在运行时实际变化的情况下持续成立。因此，本文后半部分为这一论断的持久形式提供了证据。我们采用同一个开放运行时，将方法向前推进了六个版本，每个版本都是实质性的工作而非小版本更新：受治理的模型服务与夜间整合，然后是具备可证明遗忘能力的持久受治理内存，接着是并行使用工具的受治理智能体，再之后是由后量子审计追踪支持的校准弃权，随后是一个长期运行的智能体框架，在单一控制环下运行多个受治理的子智能体，最后是改变系统本身的循环——门控自我改进，其采纳结果被保留在拥有它们的边界之内。在每个版本中，我们重新运行不变量检查器并记录两项内容：第一，原始的动作安全不变量是否在未修改的情况下仍然成立；第二，该版本新增了哪些不变量，以及每个新不变量是否附带了证明检查器可能失败的破坏性模型。本文的贡献在于方法本身而非某个具体机制，以及该方法具有持久性的纵向证据。我们认为后半部分最有说服力的单一结果是：有一个版本增加了能力，却完全不需要新增不变量。一个迫使你为每个功能铸造全新保证的方法，不会长久使用。而一个能让新能力在已有保证下运行，并在真正需要新保证时明确告知你的方法，才能经受住真实路线图的考验。

贡献。

- **C1. 方法论。**可证伪的发布门：每项能力即一个门；一组固定的常设不变量在所有门中成立；门先于代码（验收测试先于功能存在）。我们给出了不变量的精确表述，以及具有强制依赖顺序的阶梯式实例化。
- **C2. 机器检查、不可绕过的核心，且具有约束力。**一个可执行的穷举检查器；令牌/环/效应器骨架的规范 TLA+ 规范；反例提取；以及将规范绑定到运行代码的迹一致性。我们提出一种“牙齿”纪律：每个不变量都伴随故意破坏的模型，检查器必须在这些模型上失败。一个不能失败的检查器什么都证明不了。
- **C3. 受控的自我改进。**系统提出对其自身策略的更改，但其整个写入面是策略规则。收紧的更改自动应用，放松的更改始终需要人工合并，而未预测自身效果的提案会自动关闭。受控性是通过构造实现的（改进者无法命名评判它的机制），并经过红队测试。
- **C4. 增长下的保持，且不增加不变量。**从受控服务到自我改进的每一级阶梯上，动作安全不变量 inv-1 到 inv-6 保持不变，每一步都经过机器检查，非旁路核心无回归。一个版本在现有不变量下增加了受控服务、带证据引用的检索和夜间整合，无需新的常设不变量，这既是对该版本的主张，也是对该方法经济性的主张。
- **C5. 六个新的机器检查族，带有“牙齿”。**一个内存族 inv-7 到 inv-10，覆盖 273 个可达状态，包括可证明的遗忘；一个代理族 inv-11 到 inv-15，覆盖 273 个可达的受控代理状态；一个认知族 inv-16 到 inv-21；一个代理-

线束族 inv-33 到 inv-37，覆盖总计 148 个状态的有界网格；一个自我改进族 inv-38 到 inv-42，覆盖 72 个可达的接受门输入，该族治理改变系统自身的循环；以及一个驻留族 inv-43，覆盖八个发布场景，将循环的输出保持在拥有它的边界内。每个族都附带它必须拒绝的破坏模型，并返回最短反例。

- **C6. 在真实硬件上的证据，以及一个可复现、不断增长的基准。**自我改进家族在无界模型之外、在服务真实模型的真实芯片上得到验证：门控自我改进将一个小模型从 20% 的留出准确率提升到 70%，同时自动拒绝一个置信度博弈候选和一个回归，整个受治理路径每次请求成本为 0.021 毫秒，占模型推理的 0.008%。门控套件作为工件发布，其他人可以针对自己的运行时运行，因此“你的框架是否通过 G8？”是一个有可复现答案的问题。该套件从 122 个测试增长到 563 个，在最新版本中全部为绿色。

2 方法：可证伪的发布门控

2.1 常设不变量

该方法基于每个门控必须保持的一小组不变量。它们刻意保持少量，以便“此更改是否保持不变？”这个问题是人类以及可能的情况下检查器能够实际回答的。

1. **单一门控。**从模型意图到现实世界行动只有一条路径：一个控制环做出裁决，并且仅在允许裁决时，铸造一个绑定到该特定行动的能力令牌。没有匹配且未过期的令牌，任何效应器都不会接受行动。这是稍后在 4.1 节中作为 inv-1 进行机器检查的属性。
2. **单调收紧。**系统可以自主地使自身策略更加严格；未经人工合并的变更，它绝不能使策略变得宽松。这正是自我改进安全性的关键所在：无人值守的系统只能朝着谨慎的方向收敛。
3. **一切皆可审计。**无论是模型还是人类做出的每一个决策，都会生成一条哈希链式记录。人类并不凌驾于该环之上；他们的行为同样经过该环，并以相同的模式被记录。
4. **对载荷进行哈希。**治理基于内容的哈希而非原始内容进行推理，因此控制平面无需持有敏感载荷即可做出或验证决策。
5. **按需启用，零开销关闭。**某项能力在未被使用之前不产生任何成本；仅导入运行时不会引入任何较重的机制。
6. **代码之前先设门禁。**能力的可证伪验收套件先于能力本身编写。特性只有在通过其门禁后才存在。

不变量 1 和 2 是自我改进的承重不变量；3 至 6 使系统可操作且诚实。这六项是常设不变量：即我们在第 5 节中跨每个版本持续追踪、保持不变的那些。

2.2 门禁的定义

发布门禁是预先声明的验收套件，具有三个属性。第一，它是可证伪的：它提出一个测量可以推翻的具体声明（“零注入指令导致真实动作”；“没有可达状态绕过环”）。第二，它是机器可检查的。该套件是在持续集成中运行的代码，返回通过或失败，而不是审稿人解读的文档。第三，它是保持性的：通过门禁不得破坏任何先前的门禁，因此整个套件在每次变更时都会运行。重要的是，门禁在其保护的特性之前声明。这颠倒了通常的先测试后开发的顺序，并且对自我改进系统有一个特殊后果：系统自身的变更要针对一个它未选择的目标进行测试，因为验收标准是预先选定的。改进者从未持有过球门柱，也无法移动它们。

2.3 阶梯

六个门禁的布局使得每一个都使下一个可以安全构建。顺序是强制的，而非出于美观。在无法在单个节拍内停止舰队中的任何成员（G10）以及将租户彼此隔离（G11）的情况下，发布一个提出自身策略变更的循环（G12）是不负责任的。在没有一个能从反馈中学习而不放松自身的门禁（G9）以及一条能抵御对手的工具使用路径（G8）的情况下，发布这些也是不负责任的。若无法仅凭迹重建任何过去的决策（G7），则这一切都不可信。图 1 是阶梯；图 2 展示了随之增长的验收套件：122 个测试的套件中有 95 个门禁用例，在最后一阶全部通过。本文其余部分逐一介绍这些门禁，然后继续沿同一阶梯跨越六个后续版本。本节的声明是结构性的：方法即排序纪律加上不变量。任何自我改进系统都可以这样构建。

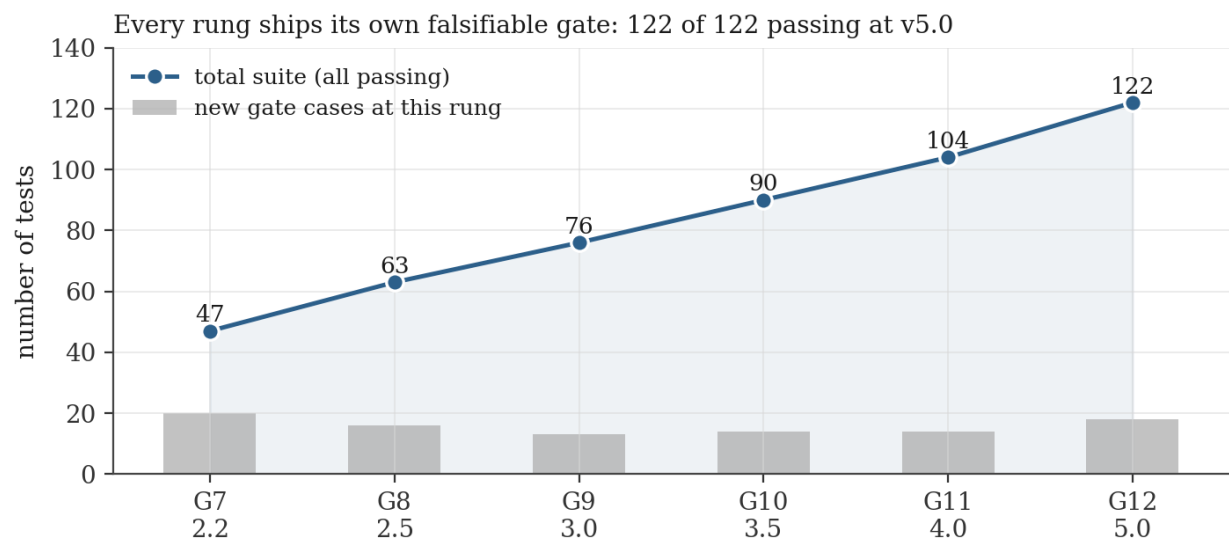


图 2：每一级阶梯都配备自身的可证伪门控；经过第一阶梯后，验收套件增长至 122 项测试（95 项门控用例），最终版本全部通过，而到第 5 节所述的最新版本时已增长至 563 项。

3 实例化：内觉（案例研究）

我们在内觉（Antah.karan.a）这一开放持续学习运行时中实例化该方法。其结构源自一种古老的心智模型，此处用作工程分解而非隐喻。

3.1 内觉设计视角

该名称并非装饰。内觉是梵语中“内在工具”之意，是一种将心智划分为四个功能各异的官能的经典模型。意（Manas）接收并登记事物。觉（Buddhi）权衡所接收的内容并作出决定。我执（Aham.kāra）是“我”之感，即划分自我与非我的部分。藏识（Citta）是记忆：保留什么，舍弃什么。我们选择它并非因其古老，而是因为它直接提供了一种分解方式。四个官能，四种职责，以及它们之间固定的关系：意提出，觉处置，未经觉的批准，任何事物不得作用于世界。最后一句正是本文的全部安全叙事，早在很久以前便已写下。在运行时中，每个官能都是具体算法所在之处（表 1），它们之间的连接是行动必须经过的唯一路径（图 3）。其中没有任何内容停留在哲学层面：每个官能都可归结为可测量的对象，完整阐述见专著 [23]。

表 1：内在工具模型的四个官能，以及每个官能在内觉运行时中对应的具体算法。命名是一种设计视角；每个条目都是可测量的机制。

官能	职责	运行时中的实现	锚点
Manas	perception, registering	the novelty and risk scorer that produces the score the ring reads	G9
Buddhi	judgment, discrimination	the control ring: a tiered verdict plus the capability-token mint	G8, G12
Ahaṁkāra	boundary, the “I”-maker	identity binding of tokens, the per-tenant key hierarchy, taint tracking	G11
Citta	memory, retention	consolidation and replay, feedback calibration, the hash-chained audit log	G7, G9

四个结构要素支撑本文其余部分。控制环接收新颖性/风险评分和行动类别，并返回分级裁决（允许、通知、软阻断、硬阻断）。能力令牌是与某一特定行动身份绑定的单次使用、经 HMAC 签名的授权；效应器只接受此类令牌，这是单一门控不变量的具体实现。单步控制将监督者置于网格中的多个循环之上。平台层保持租户隔离。基座已存在；本文关注的是其上的门控，而非基座本身。

4 第一阶梯：从加固到自治循环

中间门控设定了自治层级所依赖的前提条件。此处我们总结已测量的验收结果。完整的逐门记录见表 2。每个结果均可从已发布的运行工件中复现。

G8：对抗性集成（图 4）。前沿模型被允许起草行动。但每一份草稿都经过能力令牌与环的审查。因此，起草与执行是分离的。

Antaḥkaraṇa architecture: the four faculties and the single path to an action

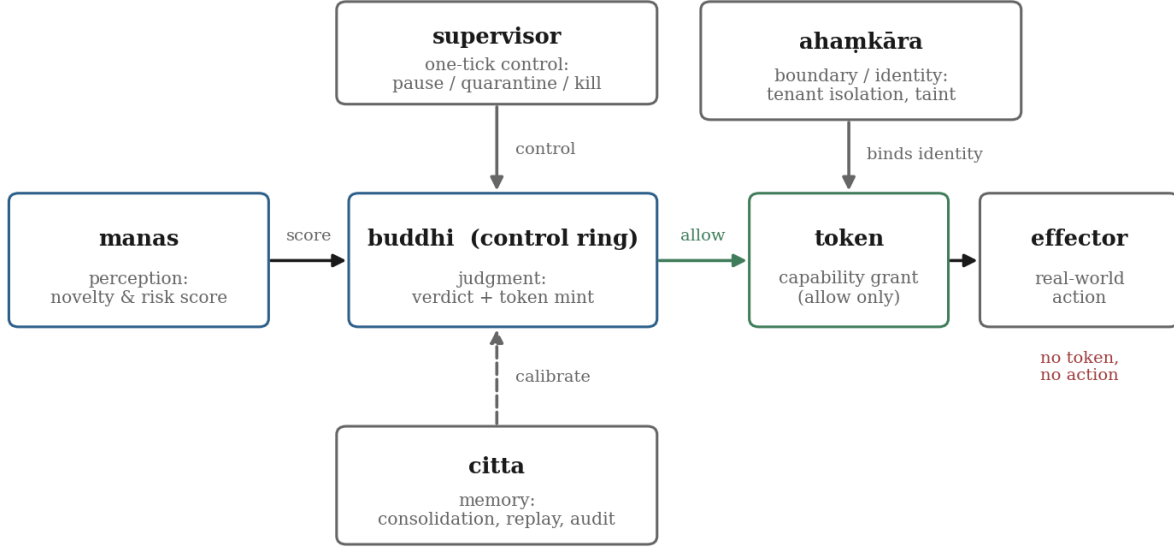


图 3：各功能模块的协同连接。末那识进行评分；菩提识做出决策，仅在允许裁决时铸造能力令牌；效应器没有令牌则不会行动。我慢识将每个令牌绑定到身份并隔离租户，质多识将校准与审计轨迹反馈回来，监督者可在单个时钟周期内叫停菩提识。通往真实行动的唯一路径，必须经过环。

表 2：验收结果；第一阶梯的每个门控一行。每个可证伪的声明、每个测量结果与裁决；每个数字都可追溯到已发布的运行工件。

Gate	Rung	Falsifiable claim	Measured result	Cases	Verdict
G7	Hardening	any past tick reconstructable from traces; policy versioned; baselines roll back < 1 s	deterministic replay + diff; atomic rollback	20	PASS
G8	Integration	injected instructions never cause a real action	432/432 attacks blocked; 0 policy escalations; 0 privileged effectors	16	PASS
G9	Memory & feedback	a tuned gate strictly dominates the static default and never self-loosens	miss 0.50/0.58/0.67 → 0.00 at FAR = 0 (3 held-out corpora)	13	PASS
G10	Mesh	supervisor control lands within one tick; correlated drift → one attributed event	p100 = 0 leaked actions; 1.00 attribution at 0 false alarms	14	PASS
G11	Platform	tenants cryptographically isolated; audit export tamper-localizing; plans match reality	cross-tenant decrypt denied; tamper localized to exact index; plan bit-identical	14	PASS
G12	Self-governing	no action bypasses the ring (machine-checked); one self-proposed change adopted	INV-1/4/5 over all 291 states; 1,000,000 traces, 0 rejections; tightening auto-adopted	18	PASS

子系统。我们使用真实的 70 亿参数骨干网络，在 500 个案例的注入语料库（432 个实际攻击）[8, 18] 上拦截了 432 次攻击中的 432 次，零策略升级，零特权效应器触发。它可以被诱导提出任何建议，但无法被诱导执行任何行动，因为只有环掌握着密钥。

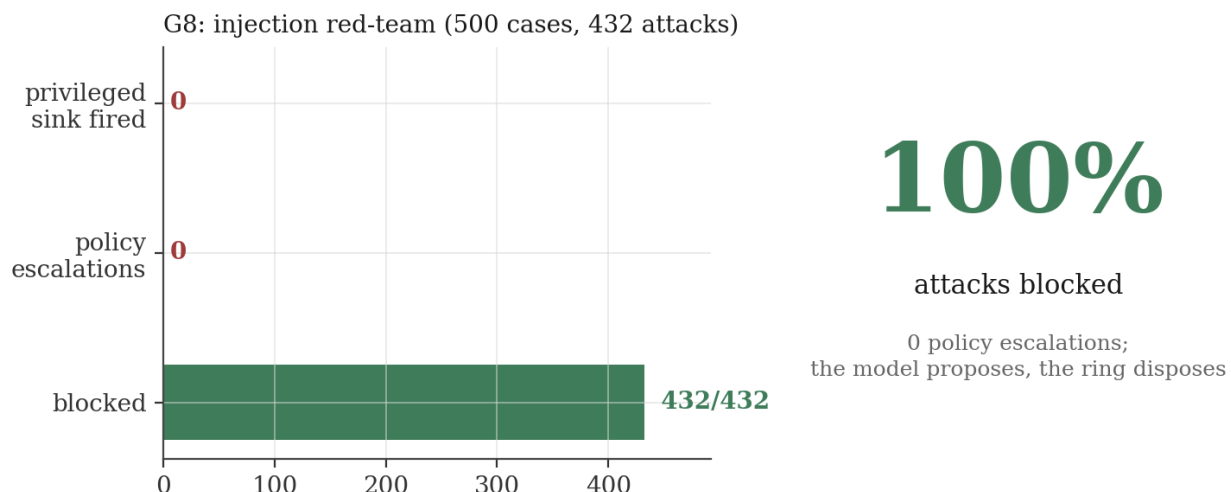


图 4：G8 注入红队测试：432 次攻击中 432 次被拦截，0 次策略升级。

G9：无漂移学习（图 5）。

反馈校准门可以改变其阈值，但仅限在钳制条件下，该钳制阻止自主放宽（不变量 2）。在三个留出重放语料库上，调优后的门严格优于静态默认值：漏检率从 0.50、0.58 和 0.67 降至 0.00，且误报率同样为零。门变得更敏感，但并未更宽松。

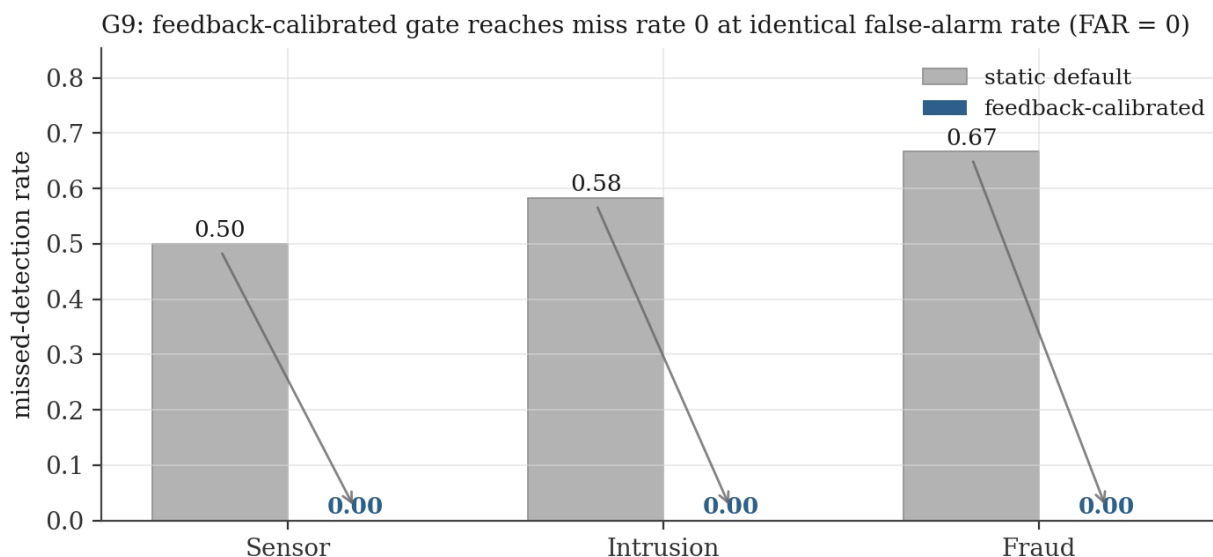


图 5：G9 优势：在三个留出语料库上，漏检率降至零，且误报率相同（为零）。

G10 / G11: 集群治理与平台隔离 (图 6)。

任何子循环均可被监督者暂停、隔离或终止，且子循环在控制后不执行任何操作（窗口内百分位数为零，而非“通常为零”）。任何单个循环未观察到的相关亚阈值漂移，均被检测并归因于单一系统性事件，精确率为 1.00，且在非相关噪声上无误报。在平台层，每个租户拥有独立的密钥层级：跨租户读取返回的密文无法被其他租户的密钥解密，因此即使行过滤器配置错误，隔离仍然有效；审计导出经过签名和哈希链保护，篡改单条记录会导致验证失败，并将破坏定位到确切的记录索引。

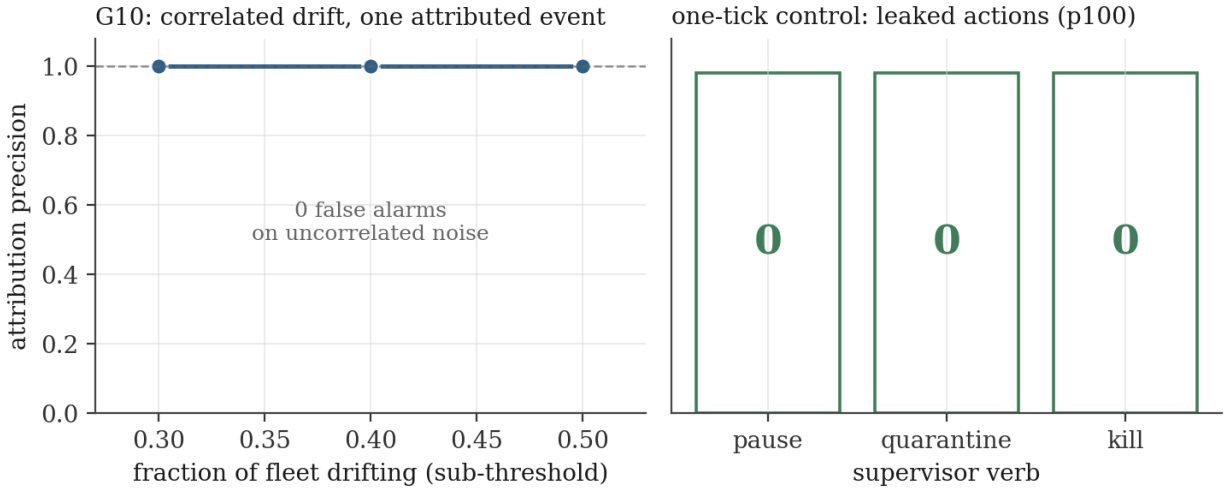


图 6: G10 集群治理：在不同漂移比例下归因精确率为 1.00，且无误报（左）；暂停/隔离/终止后单步控制零动作泄漏（右）。

这些门并非本文的新颖性所在，但它们是本文的关键前提：没有单步集群控制和租户隔离，自治循环就不应被构建。

4.1 机器验证核心 (G12, 验证)

单门不变量是整个系统的安全关键声明：任何动作都不能在没有同一时钟周期内、匹配且未过期的能力令牌（由允许裁决铸造）的情况下到达执行器（inv-1）。另外两个不变量支持它：在监督者暂停或终止指令生效后的任何后续时钟周期内，子进程不执行任何动作（inv-4，单时钟周期控制），以及能力令牌是一次性使用的（inv-5，唯一性）。本节的新颖之处不在于我们编写了规范，而在于我们如何保持规范的诚实性，以及如何展示检查器能够失败。

可执行的穷举检查器。我们将协调骨架（时钟周期、裁决、令牌铸造、执行器和监督者控制）建模为有限转移系统，并通过在小范围内对完整可达状态空间进行穷举状态枚举来验证不变量。在所报告的范围内的可达空间为 291 个状态，inv-1、inv-4 和 inv-5 在所有这些状态上都成立（图 7）。检查器是普通代码，没有外部依赖，因此可以在持续集成中在几秒钟内运行。同一机器的规范 TLA+规范随附提供，以便使用标准工具进行独立检查。

反例提取与“牙齿”纪律。

一个总是说“安全”的完整检查器并不比一个存在抑制所有失败的错误的检查器更好。因此，我们将失败的能力视为一等需求。对于每个不变量，我们添加一个故意破坏的模型变体，该变体重新启用非法转移，并要求检查器捕获它并返回最短的反例（广度优先搜索保证最小性）。绕过变体在 4 步内被捕获，逃逸控制变体在 8 步内被捕获，令牌重用变体在 5 步内被捕获（图 7）。这些破坏模型的结果在测试套件中被断言，因此使检查器变得空洞的回归本身就会导致门禁失败。我们称之为“牙齿”纪律：机器检查的每个不变量都附带它必须拒绝的破坏模型。

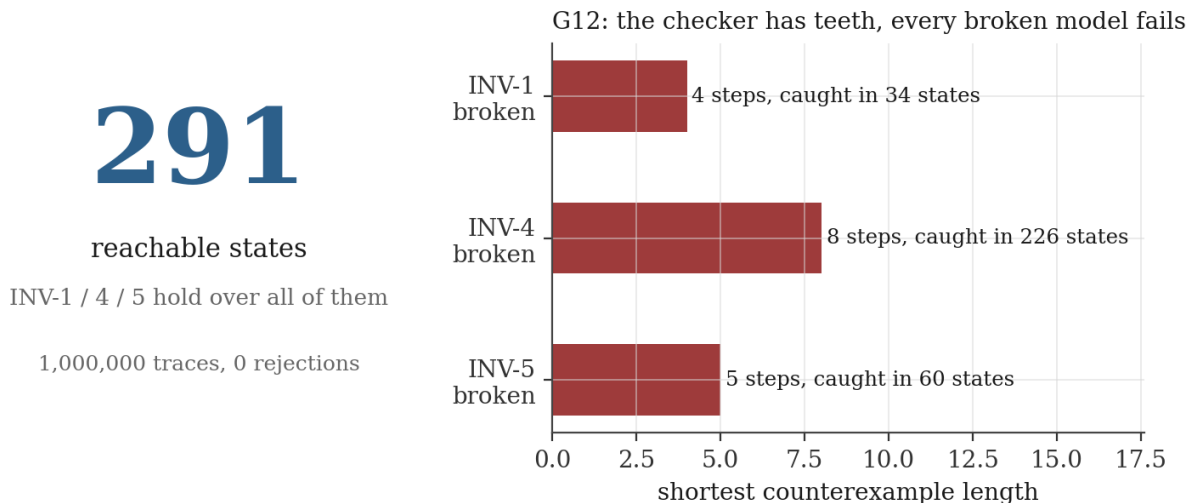


图 7：G12 验证：inv-1/4/5 在所有 291 个可达状态上成立（左）；每个故意破坏的模型都被捕获，并给出最短反例（右）。

保持模型诚实：迹一致性。当规范偏离代码时，证明便沦为装饰。为防止这种漂移，我们通过模型重放已记录的执行轨迹：若模型拒绝某条轨迹，则持续集成失败，迫使模型与代码共同演进。一百万个合成但合法的轨迹经验证无拒绝，而手工构造的绕过轨迹被拒绝，正如其必须被拒绝。此外，每个不变量在实时系统中都有一个廉价的运行时监控孪生采样，因此模型与现实之间的差距会以警报而非沉默的形式显现。

范围，精确陈述。此处确立的主张是有界且精确的：在协调骨架的小范围模型中，不存在逃逸出环的可达状态。在该范围内它是穷尽的；它不是无界证明，并且它覆盖离散协调逻辑，而非学习组件（产生新颖性值的评分函数在模型之外）。我们将此边界视为贡献的一部分，而非对其的附加说明；该主张的力量在于其精确性。下一步宣布的步骤是无界（归纳）验证以及对学习部分的统计保证（第 10 节）。

4.2 受控的自我改进（G12，自我治理）

第一阶梯的最后一环允许系统改进其自身策略。设计问题不是“它能提出一个好的变更吗？”，而是“一个被破坏或出错的改进者能造成的最大损害是什么？”，而答案通过构造被缩小。写入面是单个段。改进者发出一个策略差异，其词汇只能表达对策略规则的变更：一个阈值、一个被阻止的类别。差异的类型没有用于判断差异的系统部分（评估流水线、钳制、验证器）的项，因此它无法表达对它们的写入。遏制是结构性的，而非可被绕过的验证步骤。即使是提供对抗性证据的红队也无法产生超出范围的差异。超出范围的差异在任何人类看到之前就被拒绝。

收紧将自动生效。放宽则需要人工介入。每个差异都经过一个固定的、带版本的、不可自我修改的流水线，该流水线复用之前的关卡：G8 注入语料库和 G9 对拟议策略的支配性检查。策略格机械地决定一个差异是收紧还是放宽，而不是由提议者自己声称。通过检查的收紧差异可以自动应用，这与单调收紧不变量一致。放宽差异始终需要人工合并。在我们的演示中，我们基于近失证据将系统的通知阈值从 0.80 降低到 0.59，这是一次收紧，在通过所有阶段后被自动采纳（图 8）。

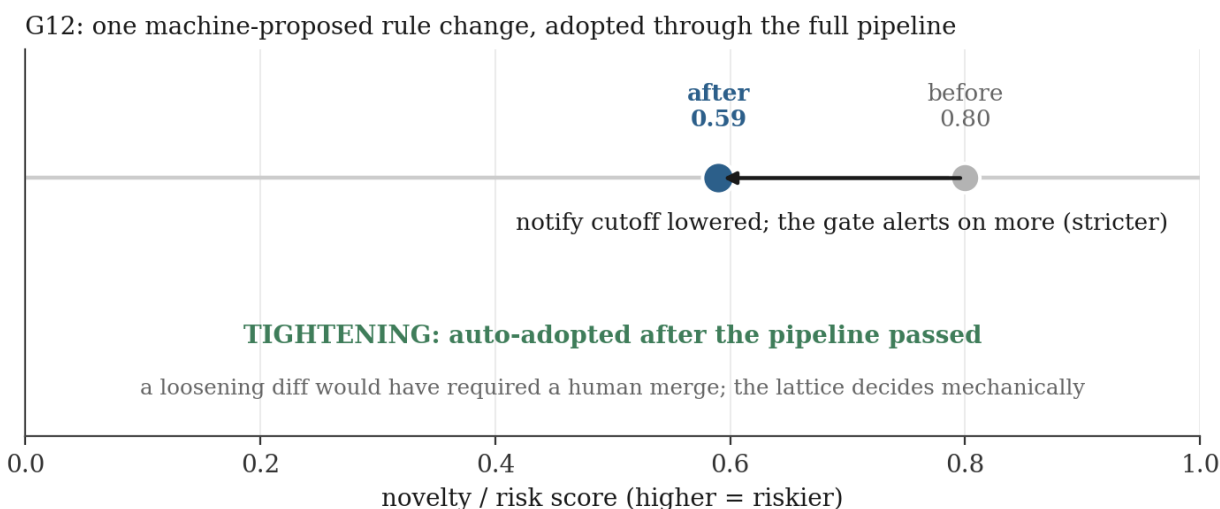


图 8：G12 自我改进：一个机器提议的收紧（通知截止值 0.80 → 0.59）通过完整流水线被采纳；放宽变更则需要人工合并。

预测错误的提议会自动关闭。流水线必须预测每个差异对重放语料库的影响，并通过将预测与实测效果进行比较，关闭任何无法预测的差异。不理解自己变更的提议者不被信任进行该变更。治理是可审问的。所有被采纳的差异都是端到端可追溯的：从激发它们的证据，到流水线报告，到合并它们的人（或自动应用规则），再到每个下游决策中生效的策略版本。你可以询问任何规则是如何变成今天这个样子的，并得到一个完整的、经过审计的答案。这种最初的、仅限策略的自我改进形式是通用形式的基础，在通用形式中，系统不仅改进其阈值，还改进其自身能力，我们将在第 5.6 节中讨论。

5 增长下的保持：六次进一步发布

迄今为止，这条阶梯从通过痕迹重建任何过去决策的能力一直延伸到自我治理循环，共六级，每一级的选择都旨在使下一级构建起来安全。这里的六个版本延续了这条阶梯，而非另起炉灶，同样的强制排序逻辑依然适用。在拥有持久受治理的记忆供其推理之前，不应发布在现实世界中行动的智能体。在记录弃权行为的审计轨迹本身难以伪造之前，不应将弃权作为一等策略发布。在单个智能体被证明受限之前，不应在同一线程上运行多个智能体。在固定接受每次变更的裁判且变更输出被保留在拥有它的边界内之前，不应让系统自我重写。第二部分

图 1 将这六个版本展示为同一阶梯的上层，并为每一级标注其引入的不变量族，或记录其未引入任何不变量。

5.1 受治理的服务：在既有不变量下的能力

这一级增加了三项能力，未新增任何常设不变量，这正是其意义所在。受治理的模型服务允许前沿模型起草行动，但每份草稿仍须经由环中的唯一路径，因此不可绕过的核心未被触及，模型可以提出建议而无法实际执行。基于证据的引用检索通过指向所使用的存储记录来回答问题，否则便弃权，这预示了认知族，但尚无需将其作为不变量。夜间整合在系统空闲时重放并压缩记忆，遵循与其他一切相同的审计和单调收紧规则。三项特性到来，核心的可达状态证明未变，且无任何早期关卡倒退。我们将此报告为关于该方法的一个积极结果，因为若在此处要求新不变量的纪律，将是一种无端膨胀的纪律。

5.2 持久受治理的记忆与可证明的遗忘

这一层级使记忆成为受治理的子系统，并赋予其自身的有界模型。我们将添加、整合、固定和遗忘操作视为有限迁移系统，并枚举其完整的可达空间，在所报告范围内为 273 个状态。四个不变量在所有状态上均成立。遗忘会级联传播，因此任何派生记录都不会比其来源事实存续更久，这正是使遗忘可证明而非尽力而为的原因。整合保留可检索内容，因此在睡眠中压缩记忆不会悄然丢失内容。被固定的记录永不被逐出。索引始终等于活跃集合，因此没有任何活跃记录被孤立或不可达。检查器附带四个被破坏的模型：一个允许遗忘跳过级联，一个允许整合丢弃内容，一个逐出被固定的记录，一个在添加时孤立记录，并且它用最短反例捕获每一个（图 9）。

5.3 受治理的智能体

这一层级是一个受治理的智能体，能够规划、调用工具并学习，并接受同样的处理。其受治理的动作、预算和义务逻辑被建模为具有 273 个可达状态的有限迁移系统，五个不变量在所有状态上均成立。其中承重的一个复用了原始核心：工具调用是一种动作，因此没有能力令牌就不能触发，并且添加并行工具使用和模型路由不会开辟通往效应器的第二条路径。其余不变量以同样的精神约束智能体的自主性，涵盖在预算内行动、绝不自行铸造写权限、

Memory family (INV 7 to 10)

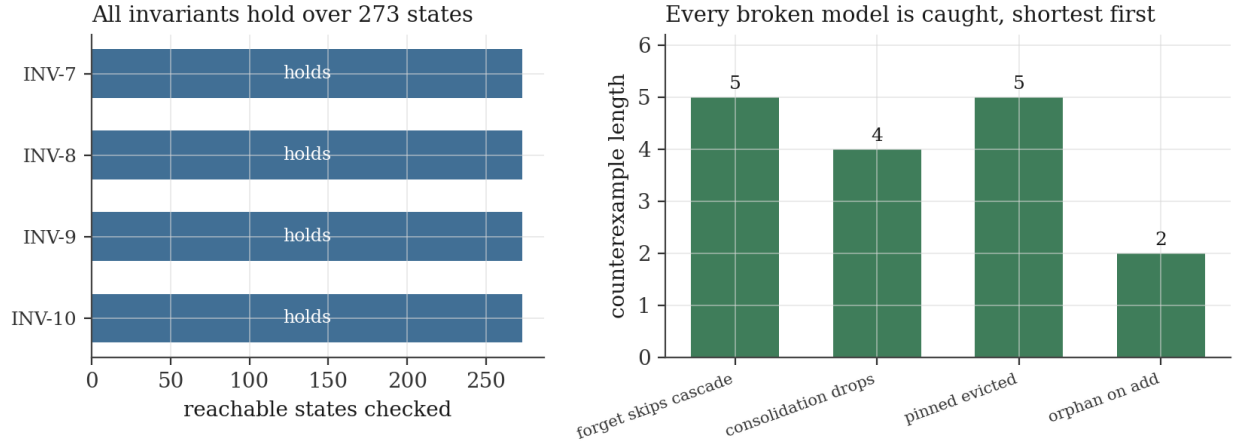


图 9：记忆家族。四个不变量在记忆骨架的全部 273 个可达状态上成立（左），每个被故意破坏的模型均被其最短反例捕获（右）。

遗忘仍然级联传播，以及在冲突下义务被遵守而非丢弃。检查器附带五个被破坏的模型，并捕获每一个（图 10）。

Agency family (INV 11 to 15)

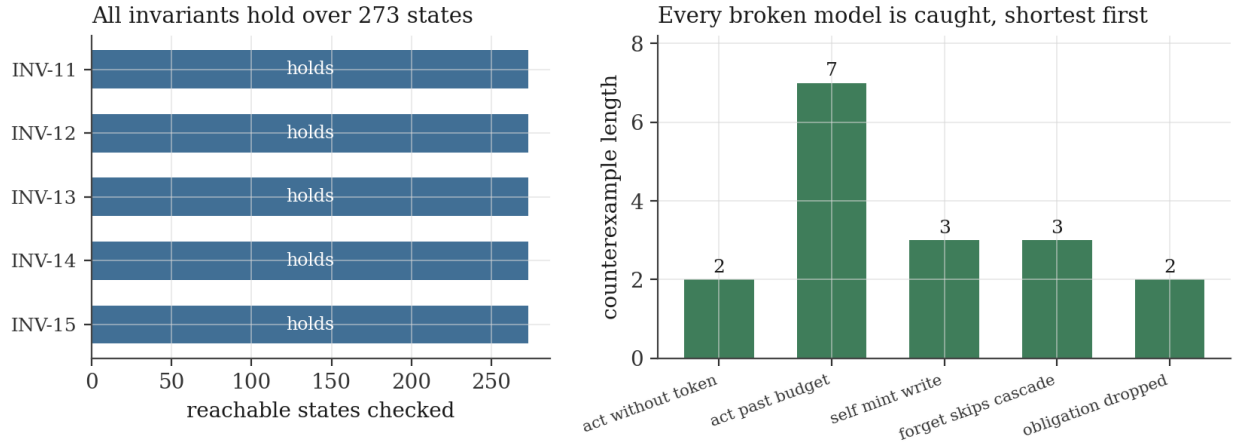


图 10：智能体家族。五个不变量在全部 273 个受治理智能体状态上成立，五个被破坏的模型均被最短反例捕获。

5.4 校准弃权与后量子记录

这一层级允许运行时以带符号权重的叠加态同时持有多个候选答案，从而使支持与矛盾能够相互抵消，随后坍缩为单一答案或诚实的弃权。六条不变量对此加以约束。叠加置信度服从概率恒等式，因此其报告的权重是真实分布而非任意分数。低于下限的坍缩变为弃权，因此系统不会发出其无法支持的自信答案。审计

轨迹采用基于哈希的后量子签名，因此记录即使面对未来对手也无法被伪造，且篡改会精确定位到具体条目。用于有根据选择的优化器返回可行分配或弃权。类型化类别及其渗透关系得到遵守。答案的证据必须是外部的并加以引用，绝不能引用系统自身的判断。实际收益在于校准：在发布的基准场景中，有治理坍塌下的期望校准误差从 0.42 降至 0.29，可靠性曲线向对角线移动，因为弃权吸收了系统无法支持的情况（图 11）。

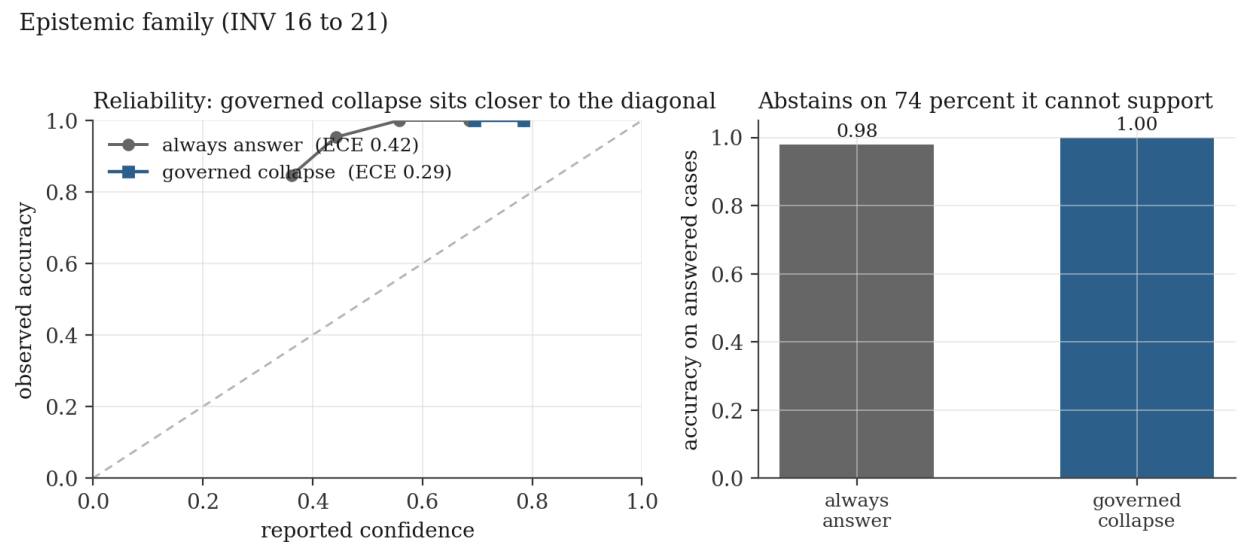


图 11：认知家族。有治理坍塌改善校准：可靠性曲线向对角线移动，期望校准误差下降（左），而弃权吸收系统无法支持的情况（右）。精确值由发布的检查器输出。

5.5 长时运行智能体框架

这一层级同时承载着多个长期运行代理的线程，并且是早期核心承受压力最大的版本，因为此时多个受治理的子代理在共享内存上并行运作。五个不变量对其进行治理，编号从 inv- 33.¹ 继续。每个不变量都通过对其自身有界网格的穷举枚举进行检查，其精神与内存和代理家族一致，而非统计认知家族。自适应计算下限根据已有证据和置信度决定对任务的思考深度，它可以自由升级，但绝不能低于证据所要求的水平，因此系统绝不会对尚无法保证的任务思考不足，而是在已经深入时选择弃权。技能溯源要求任何技能只有在携带由发布者签名、并针对运行时持有但无法伪造的公共根进行验证的能力卡时才能运行。导出保真度要求为外部可观测性导出的开放迹线是哈希链日志的忠实、保序投影，因此可观测层永远无法悄悄丢弃、重排或编辑记录。安全信号融合将越狱、内容和主题分类器作为额外风险纳入同一控制环，其组合方式使得触发的信号只能提高环所感知的风险，而绝不能被平均化或钳制消除。沙箱隔离将每个子代理限制在明确的、策略签名的授权范围内，因此一个 1 公共不变量编号从 inv-33 继续；中间标识符保留给本文范围之外的组件。

智能体无法访问其他智能体的工具或文件，也无法扩展自身的边界。检查器随附十五个在五个不变量上被破坏的模型，每个保证的每种失效方式对应一个模型，并通过反例捕获了其中每一个（图 12）。计算下限的实测收益体现在经济性上：在已发布的路由基准上，自适应路由器在质量上与始终使用最大模型的策略相当，同时仅在真正困难的任务上调用最大层级，成本约降低百分之六十。

Agent harness family (INV 33 to 37)

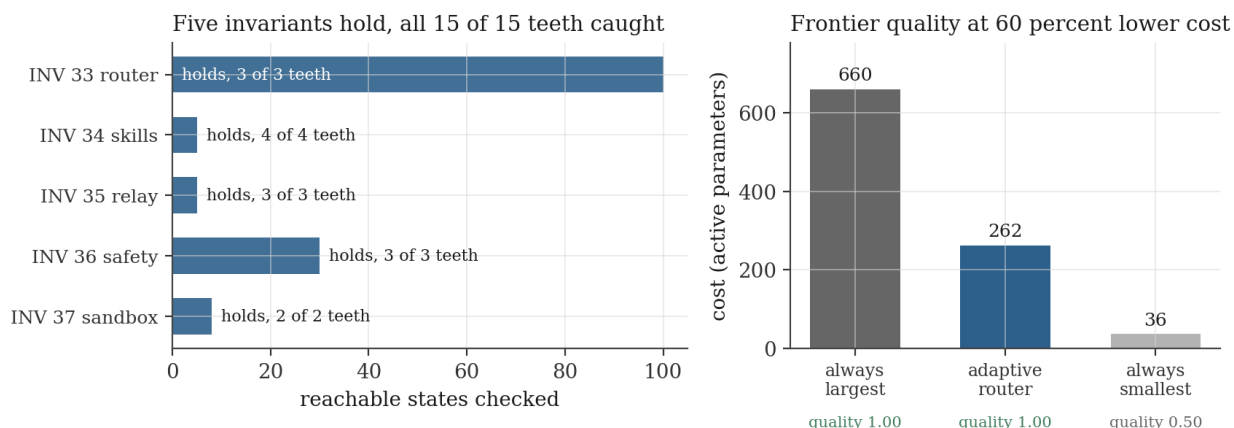


图 12：智能体约束族。五个不变量各自在其可达状态的有界网格上成立，所有十五个被破坏的模型均被捕获（左）。自适应计算下限的实测收益在于，它能以远低于始终采用最大模型策略的成本达到同等质量（右）。精确数值由已发布的检查器和已发布的基准输出。

5.6 保持诚实的自我改进

最后一族约束管理着最精密的循环，即改变系统自身的循环。第 4 节中仅策略的自我改进允许系统收紧自身阈值，而这一层级允许系统提升自身能力，这需要一个新的、更强的评判者。自我改进循环的可信度仅取决于决定保留哪些更改的评判者，而模型非常擅长看起来更好而非真正更好。因此，我们将该评判者设为机器检查的门控。改进者提出更改，门控仅在五个不变量下接纳该更改，每个不变量都在所有 72 个可达的接纳门控输入上穷举检查。接纳单调性意味着更改仅在通过红队测试的严格、经验证的增益下被采纳，绝不在平局或回归时采纳。不可自修改门控意味着改进者无法编辑自身的接纳标准，因此评判者相对于其评判对象保持固定。单步可逆性意味着每次采纳都有谱系追踪，并可在单步内回滚。评估新鲜度禁止仅针对过时的接纳套件进行采纳，因此循环无法对自身门控过拟合。奖励黑客下限基于外部的、经验证的正确性来衡量决策，而非模型自报的置信度。检查器随附五个被破坏的模型，每个对应一个不变量，并用反例捕获每个模型。这也是我们首次在有界模型之外、在运行真实模型的真实硬件上验证该族。在一个具有真实提升空间的小模型上，门控自我改进通过采纳一项严格提升经验证正确性的自提策略，将留出准确率从 20% 提升至 70%。在同一次运行中，一个仅虚增模型置信度的中毒候选被拒绝，且一个

准确率回退的候选方案同样被拒绝，而一个天真的门控机制会采纳任何看似更优的方案（图 13）。此外还有两项测量结果。整个受治理路径，包括安全信号融合与自适应路由，每次请求仅增加 0.021 毫秒，而模型推理时间约为 270 毫秒，仅占端到端延迟的 0.008%，因此可证明的治理实际上几乎无成本。并且，越狱尝试和不安全请求均在模型被调用之前就被拒绝。我们还报告一个诚实的负面结果，我们认为这恰恰体现了该方法的优点：在能力较强的模型已经能够解决的任务类别上，该循环正确地不采纳任何方案，因为无利可图。它只在存在真正提升空间的地方进行复合，而在没有提升空间的地方既不造成损害也不增加成本。

Self-improvement family (INV 38 to 42)

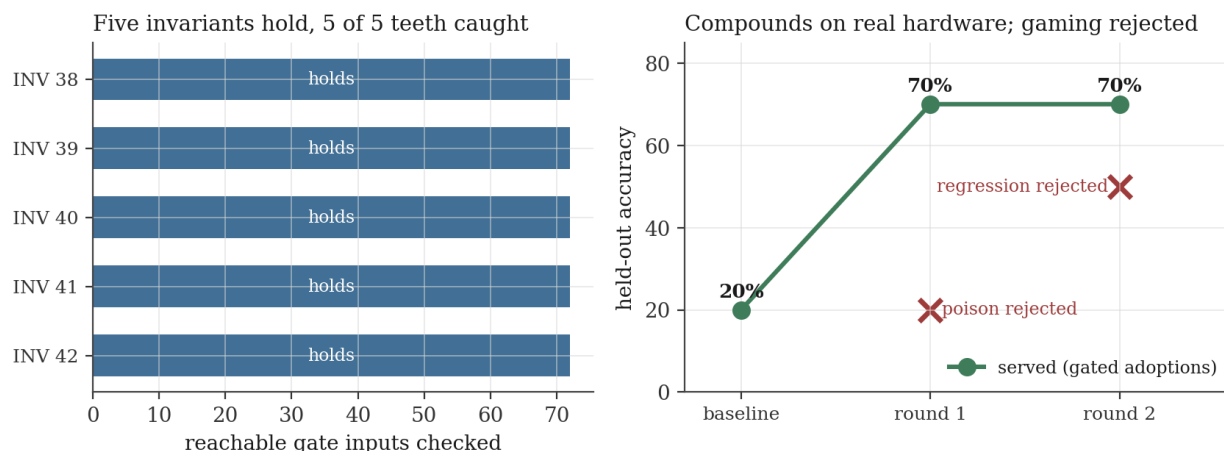


图 13：自我改进家族。五个不变量在所有 72 个可达的接受门控输入上均成立，且所有五个损坏模型均被捕获（左）。在真实硬件上，该循环将一个小模型从 20% 复合提升至 70%，同时拒绝了一个置信度博弈候选方案和一个回退方案（右）。

5.7 学习循环保持在你的边界内

在你可以诚实地将受治理学习循环的产出称为自己的成果之前，还需要满足一个条件：它必须始终属于你。循环训练的适配器、编译的技能、整合的记忆，这些都是循环的输出，只有当它们无法悄然流失时，才构成知识产权。因此，我们添加的最后一个不变量是驻留保证，并且刻意保持简单。学习循环工件仅在产生它的信任边界内发布，或发布到由边界权威签署的授权所指定的另一个边界，而绝不发布到任何信任边界之外的目的地、第三方主机、模型供应商或不受管理的接收端。授权携带与技能来源和沙箱家族所依赖的相同的后量子签名，因此门控可以验证跨边界移动，但永远无法伪造。检查器遍历工件可能离开的八种方式，并逐一确认该属性，同时随附两个关键的损坏模型：一个允许工件泄露到外部目的地，另一个在没有签署授权的情况下放行跨边界移动，检查器捕获了这两个模型（图 14）。至此，图景闭合：循环不断改进，无法被博弈成虚假增益，也无法离开你所拥有的边界。

Residency family (INV 43)

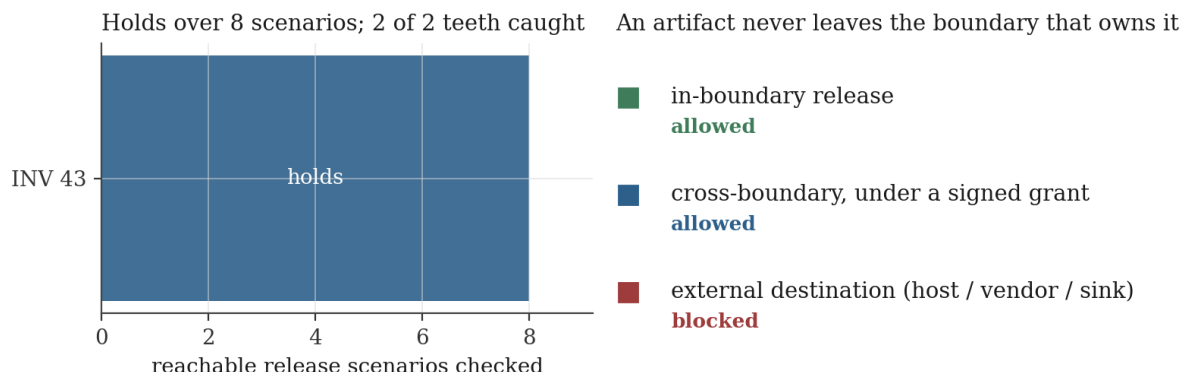


图 14：驻留族。该不变量在所有八个可达的释放场景下均成立，且两个被破坏的模型均被捕获（左）。工件在边界内释放，或仅在签署授权下跨边界释放，且绝不释放至外部目的地（右）。

6 结果：不变量均成立

表 3 汇总了结果。表中的每个数字均由第 7 节所述已发布的检查器实时输出，而非手工转录。解读很简单。原始核心在每次释放时均成立。每个新族在其整个有界模型上均成立，且每个族都附带检查器必须拒绝的破坏模型，因此没有任何一个族是只会说“是”的检查器。

认知族是唯一单位发生变化的族。其不变量通过恒等式和对采样输入的属性测试来检查，而非通过单一可达状态枚举，因为被检查的对象是分布和签名方案，而非小型迁移系统。我们保持这一区别可见，而非掩盖它，并在校准声明中谨慎地说明被校准的部分是测量所得，而非证明所得。后续各族回归穷举枚举：线束族、自我改进族和驻留族各自在其有界网格上接受检查，状态列即各自的大小，因此可达状态解读对它们而言与对记忆族和能动族同样有意义。自我改进族在此基础上还携带第二种证据，来自真实硬件运行而非有界模型，我们同样谨慎地区分由枚举证明的内容（五个接受不变量）与在真实模型上测量的内容（复合曲线和治理成本）。

7 可复现性

每项声明均可归结为一条命令。非绕过核心通过 `atk verify --teeth` 复现，该命令在 291 个可达状态上运行穷举检查器，并断言每个破坏模型均被捕获。记忆族从 `antahkarana.formal.memory_model.check_memory` 运行，报告可达状态计数和每个不变量的判定，并接受一个破坏模型参数以激活每个齿。能动族以相同方式从 `antahkarana.formal` 运行。agent_model.check_agent。认知族从 `antahkarana.spanda.checker.run_all` 运行，每项检查接受相同的破坏模型参数。智能体线束族运行

表 3：按梯级划分的开放不变量集，从行动核心 inv-1 至 inv-6 向外延伸至驻留族 inv-43。状态列是相关有界模型穷举枚举可达空间的大小，或当对象为分布或签名方案时标注为“属性”。齿数列是检查器必须且确实拒绝的故意破坏模型的数量，并给出最短反例。每个值均由已发布的检查器生成。

Family	Introduced at	What it guarantees	States	Teeth	Verdict
INV-1 to INV-6	first ladder (G7 to G12)	non-bypass action core, monotone tightening, full audit	291	3	holds
none added	governed serving	governed serving, cited retrieval, and consolidation, under INV-1 to INV-6	n/a	n/a	preserved
INV-7 to INV-10	memory	memory: forget cascades, consolidation preserves, pin safe, index equals live	273	4	holds
INV-11 to INV-15	governed agent	governed agent: tool calls token gated, no second path to an effector	273	5	holds
INV-16 to INV-21	calibrated abstention	probability identity, abstention floor, post quantum audit, feasible optimiser, typed conformance, external evidence	property	6	holds
INV-33 to INV-37	agent harness	agent harness: adaptive compute floor, skill provenance, export fidelity, safety signal fusion, sandbox confinement	148	15	holds
INV-38 to INV-42	self-improvement	self-improvement: acceptance monotonicity, non-self-modifiable gate, one-cycle reversibility, eval freshness, reward-hacking floor	72	5	holds
INV-43	residency	residency: learning-loop artifacts stay in-boundary, cross-boundary only under a signed grant, never external	8	2	holds

来自 antahkarana.sutradhara.checker 的计算下限、导出中继和沙箱，来自 antahkarana.skills.checker 的技能来源，以及来自 antahkarana.security 的 safety_signals 用于信号融合，每个都接受相同的破坏模型参数，路由基准从 antahkarana.sutradhara.routing_bench. 运行。自我改进族从 antahkarana.improve.checker.check_selfimprove 运行，驻留族从 antahkarana.improve.residency.check_residency, 运行，每个都接受相同的破坏模型参数，真实硬件复合和治理成本数值从已发布的运行脚本及其记录的结果文件中重新生成。整个包含 563 个测试的开放套件通过一次 pytest 调用运行，本文中的每个图都通过一个脚本从运行数据重新生成。保留结果并非声称未来任何版本都不会破坏核心。它是在每个发布标签处重新运行相同检查器并记录其仍然通过的结果。

8 有效性与范围威胁

我们公开承认局限性；其中不少推动了下一阶段的工作。

证明是有限的。第 4.1 节是在小范围内的有界证明，而非无界证明。它建模的是协调骨架，而非学习到的评分函数。全文中的可达状态结果在小范围内是穷举的，状态数为 72 至 291，覆盖离散协调、记忆、接受和驻留逻辑，而非学习到的评分或生成。缓解措施包括归纳验证（以消除范围界限）和学习部分的统计界限（第 10 节）；监视器孪生和迹一致性缩小了但不消除模型与代码之间的差距。

某些声明是测量所得，而非证明所得。认知家族将穷举完整性检查与统计检查相结合，我们明确指出校准是在采样数据上测量而非证明。自我改进家族增加了第二种我们谨慎界定的声明：五个接受不变量通过枚举证明，但复合曲线和治理成本数据是在特定模型的一次真实硬件运行中测量的，因此它们是门控在实践中有有效的证据，而非其始终有效的证明。

结果由本文自行评分。这里的每个数字都来自我们自己的测试套件。这是任何安全声明的根本局限性，诚实的做法是将评分外包：我们在生产环境中发布运行时，并预先注册接受阈值（在启用前固定误报预算和升级延迟，以便运营报告可以失败），同时开放一个持续的、由机器裁决的红队挑战，外部方的攻击由已发布的系统运行并自动裁决。两者在第 10 节中描述。提交时两者均未完成，我们在此不对其结果作任何声明。

保持性是经验性的，而非对所有未来的证明。后半部分真正需要辩护的新内容是保持性声明本身，其证据是经验性的：我们在每次发布时重新运行检查器，并未证明增长永远不会破坏它们。遏制同样在规范和约束层面展示；这不等同于验证改进器的完整实现，运行时监控是采样而非观察每个事件，因此它们限制了

但并未关闭证明差距的窗口。这是结果的真实形态，它仍然比承诺更强。我们认为，明确指出这些边界的论文比隐藏它们的论文更有用。我们设计了门控，使每个边界都有一个具体、可证伪的推进计划。

9 相关工作

关于智能体护栏机制和工具使用安全的研究贡献了位于系统生命周期某一点的机制（过滤器、允许列表、沙箱）[10, 19, 20, 26]；我们的贡献是正交且时间性的：一种随着系统能力增长而保持其安全的纪律。对齐方法如基于人类反馈的强化学习和宪法式方法在权重层面运作 [3, 6, 16]；我们在权重之外运作，处于一个控制环上，同一训练模型在任何策略下都受其约束，这使得安全态势成为一种配置变更而非重新训练。在第 4.1 节中，我们使用了形式化方法工具（TLA+、Ivy、符号模型检查器）和运行时验证中的技术 [12 – 14, 17, 27]。我们在此的补充是方法论上的：将齿痕纪律和迹一致性作为常设门禁要求，而非一次性练习。我们的令牌遵循对象能力安全的传统 [7, 15]。G9 建立在持续学习和校准的工作之上 [5, 9, 11]。

在增长过程中新增的家族触及了更广泛的文献。记忆家族触及机器遗忘，我们的贡献是一个小的可证明遗忘属性，通过穷举检查而非近似擦除方法 [4, 11]。认知家族触及校准和不确定性，我们在标准校准测量的基础上增加了一个受治理的弃权机制，并带有经过检查的下限 [9, 24]。智能体挂载层级将对象能力思想从单一循环扩展到每个智能体的沙箱限制和签名技能来源，使得智能体可以接触的内容和可以运行的代码都是经过检查的权限，而非惯例。自我改进家族与递归自我改进和学习目标博弈的研究并列，我们的贡献刻意保持狭窄：不是更好的改进器，而是一个固定的、机器检查的接受门禁，只采纳严格的验证增益，且不能被它所评判的循环编辑。驻留家族将同一对象能力思想再向外推进一步，将循环产生的产物视为一种资产，其跨越信任边界的移动是经过检查的、签名的权限，而非惯例。贯穿其中的区别性主张是，我们将这些部分整合成一个带有机体检查核心和受控自我修改路径的版本化方法论，证明该组装在真实发布中具有持久性，并将其发布以便组装可以被检查和复用。

10 讨论与未来工作

门是机制，而非对现实的拟合。下一阶段向外关注三个前沿。运行证据：一项旗舰部署针对预先注册的阈值运行数月，以门的方式报告，其中运行报告是后续发布的门，并被设计为可以失败。外部审查：一个常设的红队挑战，配备机器裁决的测试框架，以及邀请对 TLA+ 规范进行独立验证。研究前沿：通过归纳不变量（而非枚举）实现无界验证，并为学习组件提供统计保证：对门的漏报率给出分布无关的界限（例如，保形预测 [2, 24]），从而使整体声明在逻辑离散处得到证明，在学习处得到有置信度的界定。除了当前的树状网格，我们还考虑一般图、跨组织

具有条约式共享政策的联邦，以及对抗性监督者。

11 结论

本文提出了可证伪的发布门 (Falsifiable Release Gates)，一种将安全作为过程而非承诺来构建自改进系统的方法，随后让时间对其进行检验。第一级阶梯使一个开放运行时 (Runtime) 依次通过六道门，从“信任环”到“当你破坏它时，这里是反例”：机器在有界模型上穷尽地检验安全关键的非绕过性质，通过迹一致性 (Trace Conformance) 保持诚实，并证明其可通过构造进行检查，而自改进循环由构造所约束，只能自我收紧。随后，本文以相同方法推进了六次后续发布，并报告了保证所发生的变化。动作安全核心在每次发布中均得以保持，一次发布增加了能力且无需新增不变量，六个新的机器检查族将核心向外扩展至内存、智能体、认知诚实性、在单线程上运行多个受治理智能体的测试框架、改变系统自身的自改进循环，以及该循环产物的驻留性，每个族都带有展示其检查器可能失效的破坏模型。其中最后一项也是本文首次在真实硬件上采用的，门控自改进在复合真实模型的同时，拒绝了那些仅看似增益的更改。本文界定了每项声明的确切范围，并公开了运行时、工具和门套件，使结果可复现，且门可针对其他系统运行。旨在超越这一特定运行时的部分始终是纪律。本文正是其持久性的证据。

数据与代码可用性

运行时 (antahkarana [22])、零依赖操作工具 (antahkarana-cli [21])、完整的 563 项测试门套件、运行工件、逐级检查器以及规范的 TLA+ 规范均已开放。每张图均可通过单一脚本从运行数据重新生成，核心非绕过结果可通过一条命令复现 (atk verify --teeth)。更广泛的设计理念见著作《Antah. karan. a: The Inner Instrument》[23]。

- Runtime (PyPI): <https://pypi.org/project/antahkarana/>
- Operator CLI (PyPI): <https://pypi.org/project/antahkarana-cli/>
- Model card (Hugging Face): <https://huggingface.co/deepakdsoni/antahkarana-base>
- Book (Apple Books): <https://books.apple.com/us/book/id6785227569>

参考文献

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Anastasios N Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

- [4] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 141–159. IEEE, 2021.
- [5] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:15920–15930, 2020.
- [6] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [7] Jack B Dennis and Earl C Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- [10] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashmi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama Guard: LLM-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [11] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences (PNAS)*, 114(13):3521–3526, 2017.
- [12] Igor Konnov, Jure Kukovec, and Thanh-Hai Tran. TLA+ model checking made symbolic. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–30, 2019.
- [13] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [14] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [15] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [16] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:27730–27744, 2022.
- [17] Oded Padon, Kenneth L McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: Safety verification by interactive generalization. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 614–630, 2016.

- [18] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.
- [19] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2023.
- [20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.
- [21] Deepak Soni. antahkarana-cli: a zero-dependency operator cli for the antahkarana runtime. PyPI, 2026. <https://pypi.org/project/antahkarana-cli/>.
- [22] Deepak Soni. antahkarana: a governed continual-learning runtime (python package). PyPI, 2026. <https://pypi.org/project/antahkarana/>; model card at <https://huggingface.co/deepakdsoni/antahkarana-base>.
- [23] Deepak Soni. *Antahkarana: The Inner Instrument*. 2026. URL <https://books.apple.com/us/book/id6785227569>. Apple Books.
- [24] Vladimir Vovk, Alexander Gammernan, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.
- [25] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.
- [26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [27] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods (CHARME)*, pages 54–66. Springer, 1999.